

- * It is a ORM framework.
- * Framework: It is used to reduce work. It has Boiler-plate code [has instant code]. (In-built code).
- * ORM: Object Relational Mapping.
- * Java - Object Oriented Programming
- * SQL - Tables
- * To relate & map Java & SQL we use ORM.
- * Maven - It is a build tool. It will manage dependency.

// To create project

- * Create Maven Project → select simple project → GroupId (com.hib) → ArtifactId (project name) / Name → Finish.
- * All code we have to write in src/main/java.
- * All properties we have to write in src/main/resources.
- * All unit tests are written in src/test/java.
- * All resources belong to unit tests are written in src/test/resources.
- * All dependencies are kept in pom.xml (Extended Markup language).

- ** <dependencies> → opening tag
- </dependencies> → closing tag

- * `!--` to comment the line in maven project.

steps:

1. Session Factory (handles everything) - prepares session - to interact with database.

import jakarta.persistence.Entity;
import jakarta.persistence.Id;

```
package com.model;  
@Entity (Annotation)  
public class Employees {  
    @Id  
    private int empId;  
    private String email;  
    private double salary;  
  
    public Employees() {  
  
    }  
  
    public Employees(int empId, String email, double salary) {  
        super();  
        this.empId = empId;  
        this.email = email;  
        this.salary = salary;  
    }  
}
```

Getters & Setters, to String.

```
package com;  
public class Application {  
  
    PSVM {  
  
    }  
}
```

Steps

* Session Factory - prepares session to interact with data base.

```
SessionFactory sessionFactory = null;  
Session session = null;
```

// to configure the connections (to configure everything).

```
Configuration configuration = new Configuration();
```

```
configuration.configure("hibernate.cfg.xml");  
configuration.addAnnotatedClass(Employees.class);  
SessionFactory sessionFactory = configuration.buildSessionFactory();  
// (gives session factory)
```

```
Session session = sessionFactory.openSession();
```


** Help → Eclipse Marketplace → Hibernate tools → Enter → JBoss Tools →
 Install.
 ** New Project (rightclick) → Others → Hibernate → Hibernate Configuration File
 → Next → Next → session factory name → Database dialect → MySQL
 → Driver class → Connection URL (jdbc:mysql://localhost:3306/db)
 → Username → password (of mysql) →

* dtd - Doc Type Definition - This defines the actual hibernate.

// to interact with data base:

```

Employees emp = new Employees(12, "fhg@gmail.com", 29000);
→ Session.beginTransaction();
Session.persist(emp);
save() (insert) session.getTransaction().commit();
Sysout("inserted");
  
```

* Every class should have unique identifiers. ORM doesn't work without primary key.

18/07/2025

Class-60

package com.util;

public class HibernateUtil {

private static SessionFactory sessionFactory = null;

public static SessionFactory getSessionFactory() {

if (sessionFactory == null) {

Configuration cfg = new Configuration();

cfg.configure("hibernate.cfg.xml");

cfg.addAnnotatedClass(Employees.class);

cfg.buildSessionFactory();

return sessionFactory;

return sessionFactory;

package com;

public class Application {

public static void main(String[] args) {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

Employees emp = new Employees(12, "fgh@gmail.com", 29000);

session.beginTransaction();

session.persist(emp);

session.getTransaction().commit();

System.out.println("Inserted");

// update select

static
public void getData() {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

Employees employee = session.find(Employees.class, 1); // to see data inside table.
// primary key.

System.out.println(employee);

PSVM {

getData();

updateData();

deleteData();

}

//update.

public static void updateData() {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

Employees emp = new Employees(12, "fgh@gmail.com", 29000); session.find(Employees.class, 1);

emp.setId(1);

emp.setSalary(20000);

session.beginTransaction(); //update

session.merge(emp);

session.getTransaction().commit();

}

// Delete (Query will be written by hibernate framework)

```
public static void deleteData() {
```

```
<property name="hibernate.show-sql">true</property>
```

(It shows sql query.)

```
SessionFactory sessionFactory = HibernateUtil.getSessionFactory();
```

```
Session session = sessionFactory.openSession();
```

```
Employee emp = new Employees(12, "", 0);
```

```
session.beginTransaction();
```

```
session.remove(emp);
```

```
session.getTransaction().commit();
```

```
session.close();
```

*** Dirty Checking - autoupdate.**

Whenever we load the data from database, and change the value, we need to merge it. It will automatically merge the data. This problem is called Dirty Checking (Persistent Object).

*** States in Hibernate: 3 states.**

1. Transient state
2. Persistent state
3. Detached state.

```
Employee emp = new Employees();
```

1. **Transient state**: which is not associated with any session.
→ Dirty checking will not happen.

2. **Persistent state**: which is associated with session.

```
session.persist(emp);
```

```
session.find()
```

* Objects which are in persistent state prone to dirty checking.

3. **Detached state**:

→ Objects get detached when session is closed.

→ Dirty checking will not be happened.

package com.states;
 public class States {
 Psvm {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();
 Employees emp = new Employees(10, "dfg@gmail.com", 23000);
 // Transient state.

Session session = sessionFactory.openSession();
 session.beginTransaction();
 session.persist(emp); // ~~per~~ persistent state.
 session.getTransaction();
 session.close();
 session.setSalary(20000); // Detached state.

// After closing the resources, it is in detached state,
 dirty checking will not happened.

* // If we want to create class with other name rather than
 table name,

package com.model;
 @Entity
 @Table(name="Employees")
 public class Employee {
 }

// If we use column name other than existed, column name
 in table
 @Column(name="sal")
 private ~~string~~ double salary;

(private) what is the purpose of this?

- * If we use autoincrement in database, it will get automatically incremented Id's even if we not given Id's.
- // to know the Id in java,

@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private int empId;

Generated Value:

- * It is used to bind the id given by SQL. It will not generate value. It will not increment the value.
- Class-61 (Reverse bind the incremented value)

21/07/2025

Ex:

Create Maven Project.

package com.model;

@Entity

public class Student {

@Id, @GeneratedValue(strategy = GenerationType.IDENTITY)

private int studentId;

private String name;

private int marks;

public Student() {

public Student (String name, int marks) {
 this.name = name;
 this.marks = marks;

⇒ Constructor, getters, setters & toString method.

- * ⇒ HibernateUtil
- // to create table automatically.

* <property name = "hibernate.show_sql" > true / </property>

* <property name = "hibernate.hbm2ddl.auto" > create </property>

↑ (It will create the table)

hbm → hibernate mapping.

↓

- * Before creating, it will drop the table (existing) & create new table.

* package com;

public class Application {

PSum {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

Student st = new Student("Fayaz", 35);

session.beginTransaction();

session.persist(st);

session.getTransaction().commit();

session.close();

* Before creating, it will drop the table (existing) & create new table.

* // creates table if doesn't exist (8)
if table exist it updates the data.

<property name = "hibernate.hbm2ddl.auto" > update </property>

<property name = "hibernate.hbm2ddl.auto" > none </property>

<property name = "hibernate.hbm2ddl.auto" > create-drop </property>

(It will automatically drop the table after the session is closed)

Ex:
SQL - Structured Query Language.
HQL - Hibernate Query Language.

Ex: execute CRUD Operations using SQL HQL

package com.hql;

public class HQL {

PSVM {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

//select
SelectionQuery < student > private static void (session session) {

query = session.createQuery("select in sql

("select s from Students s", Student.class);

List < student >

list = query.list(); "From Students"

sysout(list);

PSVM {

select (session);

selectWithWhere (session);

insert (session);

update (session);

delete (session);

// where operation

private static void (session session) {

{selectWithWhere

SelectionQuery < student > query = session.createQuery (

"From student where id = ?1", Students.class);

query.set Parameter (1, 3);

List < Student > list = query.list();

sysout (list);

}

// Insert.

```
insert (session session) {
```

```
private static void select
```

```
session.createMutationQuery()
```

* // Insert, Update, Delete $\Rightarrow$ session.createMutationQuery();
session.beginTransaction();

Mutation Query query = session.createMutationQuery("Insert

into Student values(?, ?, ?);

(StudentId, name, marks)

query.setParameter(1, 1);

query.setParameter(2, "Roshini");

query.setParameter(3, 89);

query.executeUpdate();

session.getTransaction().commit();

// Update

```
private static void update (Session session) {
```

```
session.beginTransaction();
```

Mutation Query query = session.createMutationQuery("Update Student
set marks = ?1 where studentId = ?2")

query.setParameter(1, 100);

query.setParameter(2, 1);

query.executeUpdate();

session.getTransaction().commit();

}

// delete.

```
public static void delete(Session session) {  
    session.beginTransaction();  
    MutationQuery query = session.createMutationQuery("Delete from  
    Student where student Id = ?");  
    query.setParameter(1, 1);  
    query.executeUpdate();  
    session.getTransaction().commit();  
}
```

NATIVE QUERY (Original Query (SQL))

Ex: package com.nativequery;
public class NativeQueries {

psvm {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

NativeQuery<Student> query = session.createNativeMutationQuery("select * from Student", Student.class)

* For all DML - session.createNativeMutationQuery(" ");
* For all DQL - session.createNativeQuery(" ") (Insert, update, delete)

List<Student> list = query.list();
sysout(list);
}

// Named Queries

- * Execution will be faster.
- * Pre-compiled Query.

@Entity

```
@Table(name = "students")
@NamedQuery(name = "allstudents", query = "from Student")
public class Student {
    // It accepts HQL
}
```

package com.namedqueries;

public class Application {

psvm {

```
SessionFactory sessionFactory = HibernateUtil.getSessionFactory();
Session session = sessionFactory.openSession();
Query<Student> query = session.createQuery("allstudents", Student.class);
List<Student> list = query.list();
syso(list);
```

// Multiple queries.

@Entity

@Table(name = "students")

```
@NamedQueries({
    @NamedQuery(name = "all students", query = "from Student"),
    @NamedQuery(name = "update student", query = "update Student set name = ?1 where studentId = ?2")
})
```

```
public class Student {
    // (update Student set name = :
    // name where studentId = : studentId)
}
```


PSVM &

session.beginTransaction();

~~PSVM~~ &

Mutation Query

query2 = session.createNamedMutationQuery("update student",
"name"

query2.setParameter(1, "Mohanp");

query2.setParameter(2, 2);

query2.executeUpdate();

session.getTransaction().commit();

}

// Named Native Query.

@ Entity

@ Table(name = "students")

@ NamedNativeQuery -> accepts SQL.

@ NamedQueries({ @ NamedQuery ...

MAPPINGS

* In SQL - JOINS

(User -> Passport)

Types:

1. One to One (Relation has one table with one other table)
2. One to Many (Relation has one table with multiple tables ex: Order)
3. Many to One (orders - users)
4. Many to Many (ex: FLM)

Students -> Multiple Courses

Courses -> Multiple Students.

* One to One Mapping - Uni Directional relationship.

(User -> Passport)

Bi directional relationship.

(User <=> Passport)

1. OneToMany

ex:

```
package com.model;
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int UserId;

    private String name;

    public User() {
        // update (B) delete
        // If we save user, automatically, All (B) REMOVE
        // passport details also be saved. PERSIST
    }

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "pass-id")
    private Passport passport;

    public User(String name, Passport passport) {
        // ...
    }
}
```

```
package com.model;

@Entity
public class Passport {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int passportNumber;

    private String issuedPlace;

    @OneToOne(mappedBy = "passport")
    private User user;

    public Passport() {
        // ...
    }

    public Passport(int passportNumber, String issuedPlace) {
        // ...
    }
}
```

Getters, setters & toString, constructors.

package com.mapping;

public class OneToOneMapping {

psvm {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

User user = new User("Fayaz", new Passport(1000, "1234567890"));

session.beginTransaction();

session.persist(user);

session.getTransaction().commit();

* Cascading: Used to send the data.

// to get back the data (select Operation).

psvm {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

User user = session.find(User.class, 2);

syso(user);

// delete

psvm {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

session.beginTransaction();

session.remove(new User(2, "Fayaz", new Passport(1000, "1234567890")));

session.getTransaction().commit();

// Insert

```
psvm {
    session.beginTransaction();
    User user = new User("Pranneth", new Password(1001, "Hyd"));
    session.persist(user);
    session.getTransaction().commit();
}
```

```
User user = new User("Pranneth");
Passport passport = new Password(1001, "Hyd");
user.setPassport(passport);
session.beginTransaction();
passport.setUser(user);
session.getTransaction().commit();
```

```
Passport passport = session.find(Passport.class, 1001);
```

```
System.out.println(passport);
```

```
System.out.println(passport.getUser());
```

⇒ Bi directional Mapping.

* Composite Primary Key (Having two & more primary keys.

ex:

```
package com.model;
```

```
@Entity
public class Car {
```

```
    private String CompanyName;
```

```
    private double price;
```

```
    @Embedded Id
```

```
    private CarId carId;
```

```
    public Car() {
```

```
    }
```

Constructor, getters & setters, toString method.

package com.model;

@Embeddable

public class CarId {

private int id;

private String engineNumber;

public CarId() {

}

Constructors, getters, setters & toString.

package com.embeddable;

public class Application {

psvm {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

// Car car = new Car("Maruthi Suzuki", 1000000);

session.beginTransaction();

Car car = new Car(new CarId(1, "thebst628346"),

"Maruthi Suzuki", 1000000);

session.persist(car);

session.getTransaction().commit();

// select operation.

session.find(Car.class, new CarId(1, "thebst628346"));

* If we get StackOverflow exception in SQL, we can handle with annotation - @JsonBackReference.

2. OneTo Many:person $\rightarrow$ Orders.3. ManyTo One:Orders $\rightarrow$ Person* Person class

```

package com.model;

@Entity
@Table(name = "Persons")
public class Person {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int personId;

    private String name;

    private String mobileNumber;

    private List<Order> orders;

    public Person() {
        fetch = FetchType.EAGER;
    }

    public Person(String name, String mobileNumber) {
        super();
        this.name = name;
        this.mobileNumber = mobileNumber;
    }

```

Constructor, getters & setters, toString.

* Order class

```

package com.model;

@Entity
@Table(name = "orders")
public class Order {
    @Id
    private int orderId;

    private String productName;

    private int quantity;

    private Person person;

    public Order() {
        @ManyToMany(mappedBy = "orders")
        @JoinColumn(name = "person_id")
    }

```

Constructor, getters & setters, toString.

package com.mapping;

public class OneToManyMapping {

psvm {

SessionFactory sessionFactory = HibernateUtil.getSessionFactory();

Session session = sessionFactory.openSession();

Person person = new Person("Sandeep", "1234567890");

Order order = new Order(1, "Redmi Mobile", 1);

Order order2 = new Order(2, "Dell Laptop", 1);

order.setPerson(person); order2.setPerson(person);

List<Order> orders = new ArrayList<Order>();

orders.add(order);

orders.add(order2);

person.setOrders(orders);

session.beginTransaction();

session.persist(person);

session.getTransaction().commit();

// select

Person person = session.find(Person.class, 1);

sysout(person);

sysout(person.getOrders());

* Fetch types: 2

1. Lazy loading - when we load a person's data, it will not try to fetch Order data.

(fetch = FetchType.LAZY)

2. Eager Loading - When we load person's data, it will automatically load order data also.

(fetch = FetchType.EAGER)

// Update

```
session.beginTransaction();
```

```
Person person = session.find(Person.class, 1);
```

```
person.setName("Venkat");
```

```
session.merge(person);
```

```
session.getTransaction().commit();
```

// We can update order details also

```
session.beginTransaction();
```

```
Person person = session.find(Person.class, 1);
```

```
person.setName("Venkat");
```

```
List<Order> orders = person.getOrders();
```

```
Order order = orders.get(0);
```

```
order.setProductName("Realme Mobile");
```

```
session.merge(person);
```

```
session.getTransaction().commit();
```

4. Many To Many

Ex:

```
package com.model;
```

```
@Entity
```

```
public class Trainee {
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private int traineeId;
```

```
    private String name;
```

```
    @ManyToMany(mappedBy = "trainees")
```

```
    private List<Course> courses;
```

```
    public Trainee() {}
```

```
    public Trainee(String name) {
```

```
        this.name = name;
```

```
    }
```

* Create new Table by taking (Composite Primary Key).

```
traineeId CourseId;
```

```
List<Trainee> trainees = new ArrayList<>();
```

```
List<Course> courses = new ArrayList<>();
```

```
List<Trainee> trainees = new ArrayList<>();
```

```
List<Course> courses = new ArrayList<>();
```



```
package com.model;
```

```
@Entity
```

```
@Table(name = "courses")
```

```
public class Course {
```

```
@Id
```

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private int courseId;
```

```
private String courseName;
```

```
@ManyToMany
```

```
@JoinTable(name = "trainees_courses",  
joinColumns = { @JoinColumn(  
name = "traineeId",
```

```
inverseJoinColumns =  
{ @JoinColumn(name = "courseId") }  
)
```

```
public Course() {  
}
```

```
constructors, getters & setters, toString
```

```
package com.mappings;
```

```
public class ManyToManyMapping {
```

```
psvm {
```

```
SessionFactory sessionFactory = HibernateUtil.getSessionFactory();
```

```
Session session = sessionFactory.openSession();
```

```
// Insert  
session.beginTransaction();
```

```
Trainee t1 = new Trainee("Deepak");
```

```
Trainee t2 = new Trainee("Jagadeesh");
```

```
Trainee t3 = new Trainee("Prasanth");
```

```
Course c1 = new Course("JFS");
```

```
Course c2 = new Course("Adv Java");
```

```
Course c3 = new Course("AI");
```

```
List<Trainee> trainees = new ArrayList<Trainee>();
```

```
List<Course> courses = new ArrayList<Course>();
```

```
List<Trainee> trainees2 = new ArrayList<Trainee>();
```

```
List<Course> courses2 = new ArrayList<Course>();
```


trainees.add(t₁);

trainees.add(t₂);

trainees2.add(t₃);

Courses.add(c₁);

Courses.add(c₂);

Courses2.add(c₃);

t₁.setCourses(courses);

t₂.setCourses(courses);

t₃.setCourses(courses2);

c₁.setTrainees(trainees);

c₂.setTrainees(trainees);

c₃.setTrainees(trainees2);

Session.persist(t₁);

Session.persist(t₂);

Session.persist(t₃);

Session.getTransaction().commit();

}

// select

Trainee
trainee = session.find(Trainee.class, 1);
sysout(trainee);
sysout(trainee.getCourses());